



Activity File: Exploiting the `tar` Command with the Checkpoint and Wildcard Options

In this exercise, you'll play the part of a penetration tester hunting for vulnerabilities in a target system's backup procedures.

- You'll use the `wildpwn.py` tool to verify that the system's current configuration is indeed vulnerable to this kind of privilege escalation attack.
- Then, you'll research and suggest two mitigation strategies to protect the server in the future.

Instructions

1. Using the following credentials, log in as `jane` and use `sudo -l` to verify that you have no `sudo` rights:
 - Username: `jane`
 - password: `password`
2. Next, we'll look for automated `tar` backup files that can be exploited.
 - As `jane`, navigate to your `~/Documents` directory in the terminal. Are there any archive files present?
 - What might the name of the file indicate? Use `ls -lat` to check the ownership of the archive. Why wouldn't this file belong to `jane`?
 - Delete the archive file. You'll now need to wait a minute or two to test to see if the archive file comes back. If the archive file comes back, this means it can be potentially exploited with a `wildcard` attack.
 - Now that we've confirmed there is a regenerating archive file that is potentially backing up the contents of `jane`'s `Document` directory, we'll continue with the following steps:
 - Downloading the `wildpwn` exploit tool.
 - Running it to create payloads.
 - Waiting for the automated `tar` backup system task also archives the malicious payload files, thus activating our exploit.
 - Note that there is a subdirectory, `ExploitTar`, that has been created for your pen-testing tasks. Use this folder to run `wildpwn` in.
3. Download the [wildpwn.py](#) attack tool to your `~/Documents/ExploitTar` directory. Do this from the command line, not the browser.
 - **Hint:** Use `wget` or `curl`.

- Navigate to and check the contents of this directory before we execute the python script.
 - 4. Use `wildpwn.py` to generate the malicious `tar` payload files within the `/home/jane/Documents/ExploitTar` directory, which is expected to be frequently backed up.
 - **Remember:** The exploit becomes staged when the system's automated backup task archives Jane's documents into the `tar` file that we found earlier.
 - If you are unsure how to use this file: After downloading the script, run `python wildpwn.py -h` to look up usage. If you are stuck, ask your instructor or TA for the proper command.
 - Run `ls` to check the directory for our newly created files.
 - As the regular user `jane`, you just planted malicious files that will create the exploit file when they are automatically backed up by the system.
 - 5. Now we wait.
 - After waiting a minute or two, re-run the `ls` command but with the proper options to see the new hidden subdirectory.
 - Navigate into that subdirectory and list its hidden file. This is the one you'll want to run to execute the `wildcard attack exploit`.
 - 6. Time to run the exploit.
 - While still in the hidden subdirectory, execute the hidden file within the terminal.
 - What happened to your command prompt that confirms this exploit worked?
 - 7. As `root`, make `jane` a superuser with `sudo` access.
 - Add `jane` to `/etc/sudoers` by running `visudo`. This will allow our user `jane` to run commands with superuser privileges. Also, specify the `NOPASSWD` rule for this user.
 - If you're not sure how to do this, there is a similar user setup under `root`'s **specifications**.
 - 8. Let's test our new privileges and then cover our tracks.
 - Log back in as `jane` and verify that you can run commands with `sudo`.
 - Then, remove the entire `ExploitTar` directory to cover your tracks.
 - 9. Congratulations! You have successfully executed what is known as a `tar wildcard attack`! This is a more discreet attack that can be carried out on a Unix-based operating system with `tar` installed.
 - After verifying that you can escalate privileges and modify the system to catastrophic effect, identify two ways to prevent this attack in the future.
-